Mitali Bhatia
D15C 16
MLDL

Experiment No 07

Aim: Build an Artificial Neural Network (ANN) using Keras/TensorFlow

1. Dataset Source

The dataset used in this experiment is the **Pima Indians Diabetes Dataset**.

- **Source:** Kaggle
- **Dataset Name:** Pima Indians Diabetes Dataset
- **Link:** https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database

This dataset is widely used for binary classification problems in machine learning.

2. Dataset Description

The dataset contains medical details of female patients and is used to predict whether a person has diabetes or not.

- **Number of instances:** 768
- **Number of features:** 8 input features + 1 target

Features:

- Pregnancies: Number of times pregnant
- Glucose: Blood glucose level
- BloodPressure: Blood pressure value
- SkinThickness: Skin fold thickness
- Insulin: Insulin level
- BMI: Body Mass Index
- DiabetesPedigreeFunction: Genetic influence factor
- Age: Age of the patient

Target Variable:

- **Outcome**
  - 0 → No Diabetes
  - 1 → Diabetes

Dataset Characteristics:

- Numerical dataset
- No categorical values
- Binary classification problem
- Some values may be zero (treated as missing in real scenarios)

---

3. Mathematical Formulation of the Algorithm

Artificial Neural Network (ANN) is inspired by the human brain and consists of layers of neurons.

Neuron Operation:

Each neuron performs:

$$\boxed{\begin{gathered} z = w_1 x_1 + w_2 x_2 + \ldots + b \\ a = f(z) \end{gathered}}$$

Where:

- x = input features
- w = weights
- b = bias
- f(z) = activation function

Activation Function (Sigmoid):

$$\boxed{\sigma(z) = \frac{1}{1 + e^{-z}}}$$

Loss Function (Binary Cross Entropy):

$$\boxed{Loss = -\frac{1}{n} \sum [y \log(p) + (1 - y) \log(1 - p)]}$$

Optimizer:

- Adam optimizer is used to update weights and reduce loss.

---

4. Algorithm Limitations

- Requires large data for better performance
- Training can be time-consuming
- Works like a “black box” (hard to interpret)
- Sensitive to feature scaling
- Can overfit if not properly tuned

---

5. Methodology / Workflow

5.1 Overall Workflow

1. Load dataset
2. Separate features and target
3. Split data into training and testing sets
4. Apply feature scaling
5. Build ANN model
6. Compile model
7. Train model
8. Make predictions
9. Evaluate performance
10. Plot graphs

Flow:

Dataset → Preprocessing → Train-Test Split → Scaling
→ ANN Model → Training → Prediction → Evaluation → Graphs

5.2 ANN Workflow

1. Load diabetes dataset
2. Separate input features (X) and target (y)
3. Split dataset into training and testing sets
4. Apply StandardScaler for normalization
5. Build ANN with:
    - Input layer
    - Hidden layers (ReLU activation)
    - Output layer (Sigmoid)
6. Compile model using Adam optimizer and binary cross-entropy loss
7. Train model using epochs and batch size

8. Predict outcomes on test data
9. Evaluate using accuracy, confusion matrix, and classification report
10. Plot accuracy and loss graphs

---

6. Performance Analysis

The model performance was evaluated using the following metrics:

Accuracy

- Measures how many predictions are correct
- In this model, accuracy is printed after testing

Confusion Matrix

- Shows:
    - True Positive
    - True Negative
    - False Positive
    - False Negative

Classification Report

- Includes:
    - Precision
    - Recall
    - F1-score

Graphs

- Accuracy vs Epochs → shows learning progress
- Loss vs Epochs → shows error reduction

Interpretation

The ANN model achieves good accuracy and is able to classify diabetic and non-diabetic patients effectively.

---

7. Hyperparameter Tuning

The following hyperparameters were used and adjusted:

- **Epochs (100):** Number of training iterations
- **Batch size (32):** Number of samples per update
- **Hidden layers (8 and 6 neurons):** Controls model complexity

- **Activation functions:** ReLU (hidden), Sigmoid (output)
- **Optimizer:** Adam

Impact of Tuning:

- Improved model accuracy
- Better convergence
- Reduced loss over time
- Balanced training and validation performance

CODE:

```
# Install (runs only first time)
!pip install tensorflow pandas scikit-learn matplotlib seaborn -q

# ==============================
# 1. Import Libraries
# ==============================
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# ==============================
# 2. Load Data
# ==============================
df = pd.read_csv('diabetes.csv')

# ==============================
# 3. Preprocessing
# ==============================
```

```python
X = df.drop('Outcome', axis=1)
y = df['Outcome']

# ==============================
# 4. Split Data
# ==============================
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ==============================
# 5. Feature Scaling
# ==============================
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# ==============================
# 6. Build ANN
# ==============================
model = Sequential([
    Dense(8, activation='relu', input_shape=(X_train.shape[1],)),
    Dense(6, activation='relu'),
    Dense(1, activation='sigmoid')
])

# ==============================
# 7. Compile Model
# ==============================
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# ==============================
# 8. Train Model
# ==============================
```

```python
history = model.fit(
    X_train,
    y_train,
    batch_size=32,
    epochs=100,
    validation_split=0.2,
    verbose=1
)

# ==============================
# 9. Evaluate Model
# ==============================
y_pred = model.predict(X_test)
y_pred = (y_pred > 0.5)

print("\n--- RESULTS ---")
print(f"Accuracy: {accuracy_score(y_test, y_pred)*100:.2f}%")

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# ==============================
# 10. Graphs
# ==============================
plt.figure(figsize=(14,5))

# Accuracy graph
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train')
plt.plot(history.history['val_accuracy'], label='Validation')
plt.title('Accuracy over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

# Loss graph
plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Train')
```

```
plt.plot(history.history['val_loss'], label='Validation')
plt.title('Loss over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

plt.show()

# ==============================
# 11. Confusion Matrix
# ==============================
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')

plt.title('Confusion Matrix')
plt.xlabel('Predicted (0: No Diabetes, 1: Diabetes)')
plt.ylabel('Actual (0: No Diabetes, 1: Diabetes)')

plt.show()
```

OUTPUT:

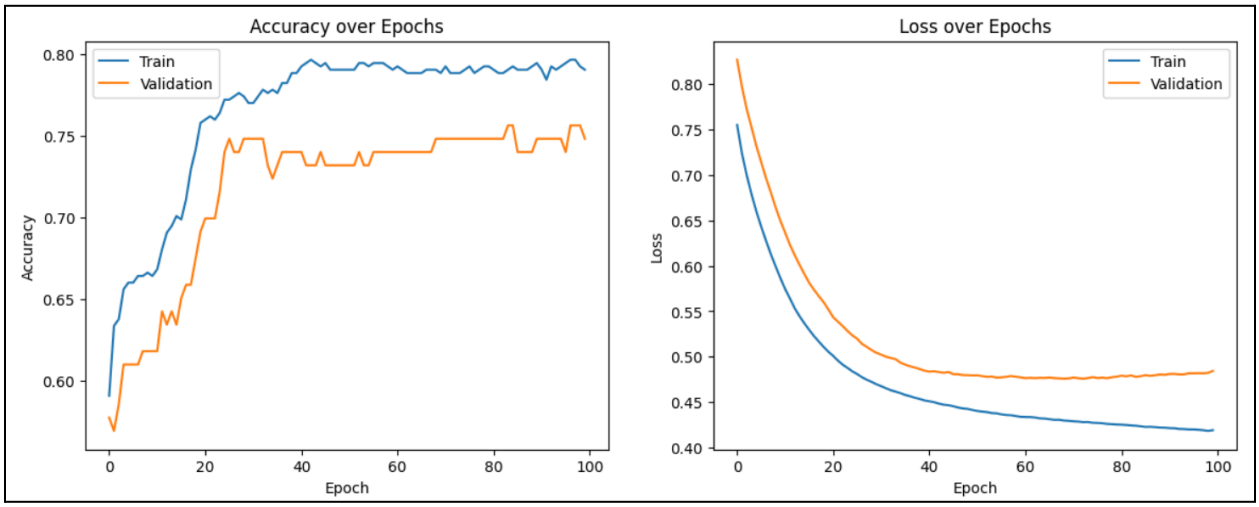
```
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7963 - loss: 0.4199 - val_accuracy: 0.7561 - val_loss: 0.4821
Epoch 98/100
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7963 - loss: 0.4193 - val_accuracy: 0.7561 - val_loss: 0.4819
Epoch 99/100
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7923 - loss: 0.4186 - val_accuracy: 0.7561 - val_loss: 0.4824
Epoch 100/100
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7902 - loss: 0.4194 - val_accuracy: 0.7480 - val_loss: 0.4844
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
```

```
--- RESULTS ---
Accuracy: 74.68%

Classification Report:
              precision    recall  f1-score   support

           0       0.80      0.81      0.80        99
           1       0.65      0.64      0.64        55

    accuracy                           0.75       154
   macro avg       0.72      0.72      0.72       154
weighted avg       0.75      0.75      0.75       154
```



Conclusion

In this experiment, an Artificial Neural Network was successfully implemented to predict diabetes using medical data. The model was trained using scaled features and evaluated using accuracy, confusion matrix, and classification report. The results show that ANN can effectively handle classification problems and provide reliable predictions.